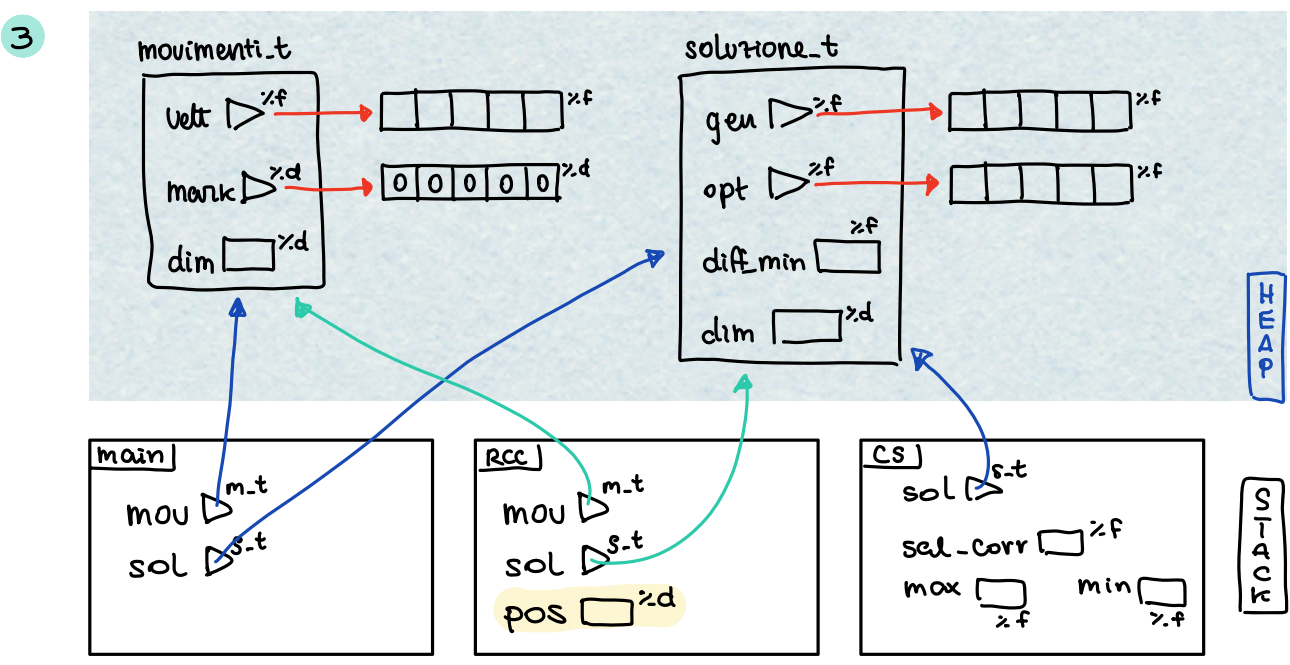
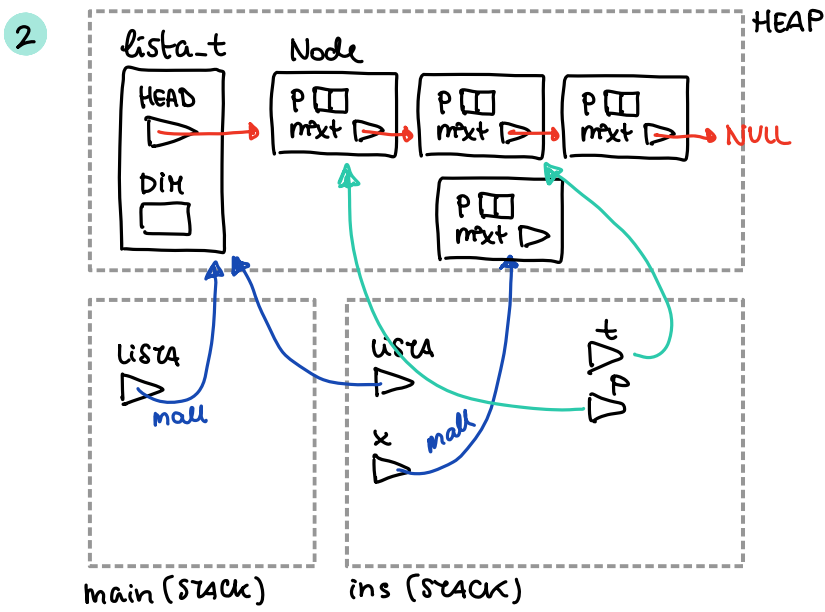


2014-02-06

12 Pt



18 pt

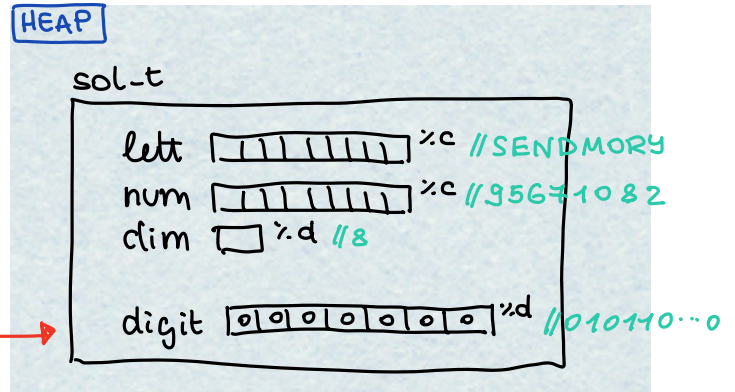
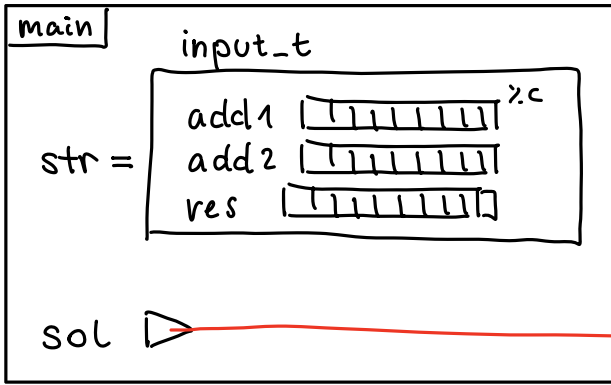
digit
① 1 2 3 4 5 6 7 8 9

9 5 6 7		
SEND 10		
1 0 8 5		
MORE 10	→ sol → left	SEND MORY
1 0 6 5 2		
MONEY 10	sol → num	[i + '0']

1	2	3
A	B	C
1	2	3
A	B	C
2	4	6
B	D	F

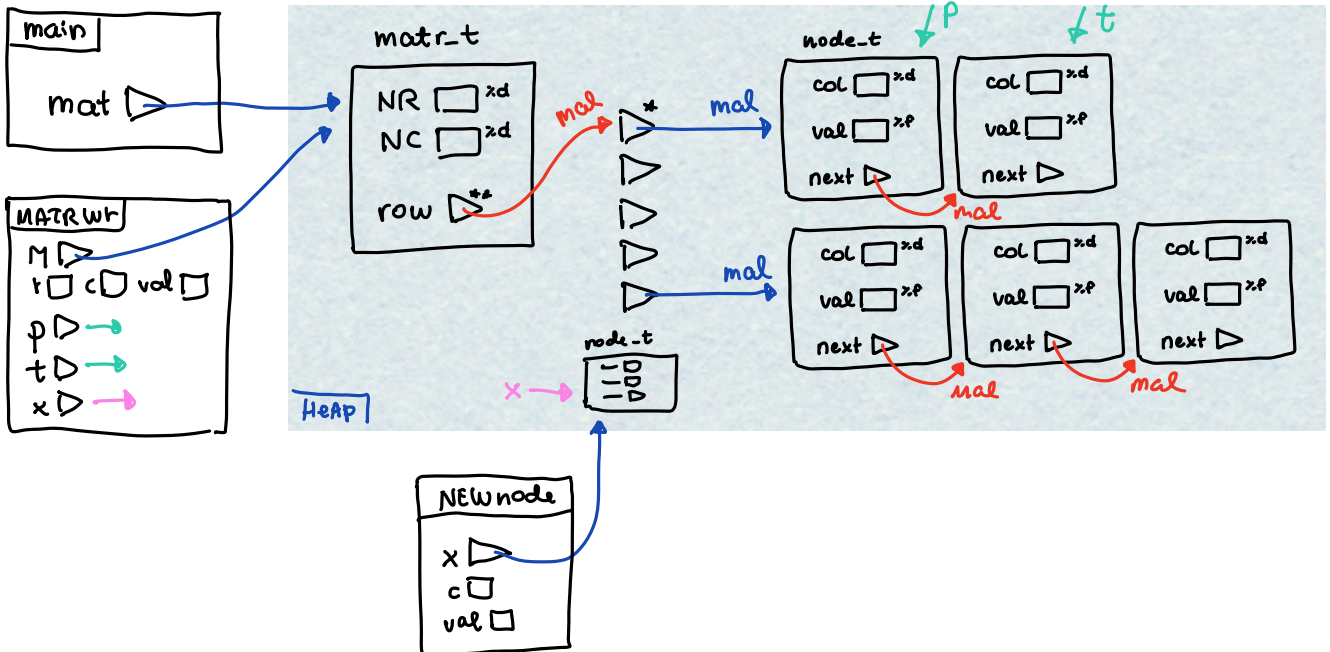
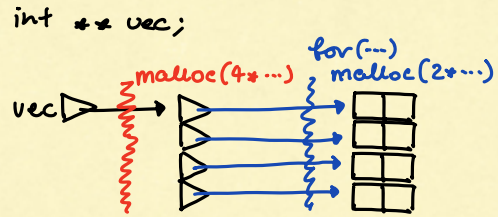
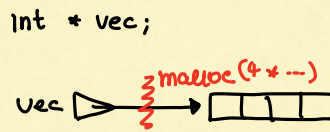
Pos

sol $\rightarrow$ digit $\left[\begin{array}{cccccccccccc} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \end{array} \right]$



2014-02-24

2



3

Voc

A	E	I	O	U
---	---	---	---	---

 %c

mark

0	0	0	0	0
---	---	---	---	---

 $\stackrel{z}{=} d \quad \{0, 1, 2, \dots\}$

Posti disp

$$\boxed{\text{f-pos}} > \text{sum}_0$$

condizione
di selezione

str

A E I O U A A | \0 %c
N+1

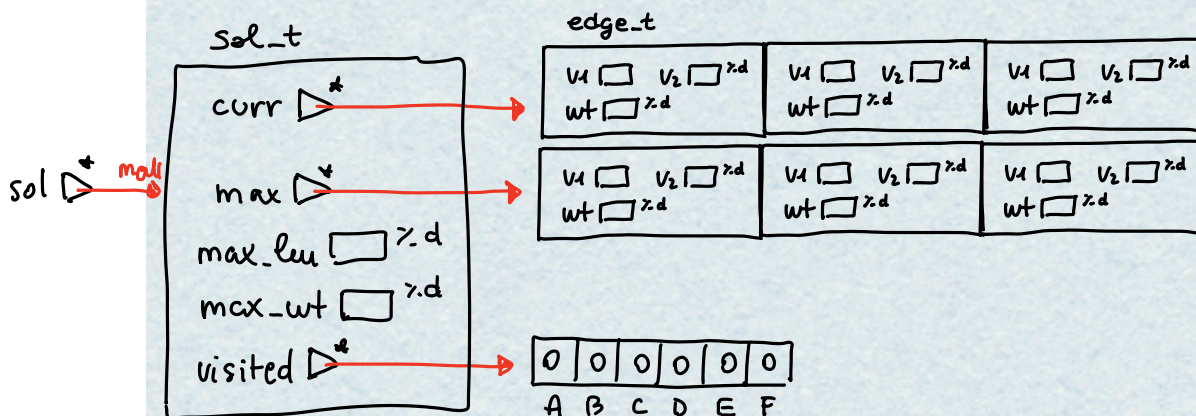
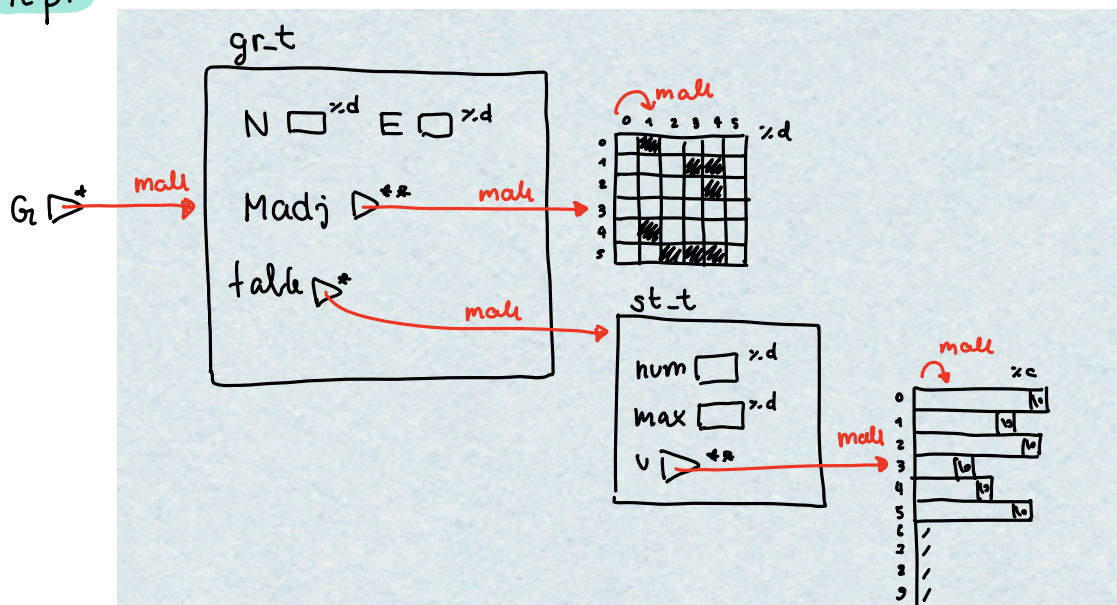
// PERMUTATIONI SEMPLICI

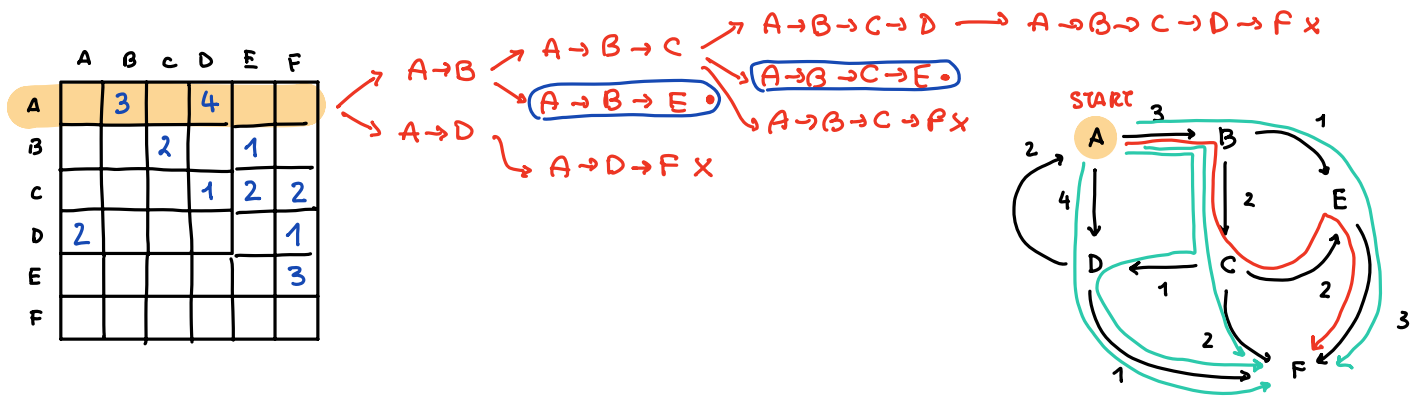
```
for (i=0; i<5; i++) {
    if (mark[i] == 0) {
        mark[i] = 1;
        sol[pos] = val[i];
        R -- (pos+1);
        mark[i] = 0;
    }
}
```

// DISP. CON RIPETIZIONE

```
for (i=0; i<5; i++) {
    if ((N-pos) > sum_0 || mark[i] == 0) {
        str[pos] = voc[i];
        mark[i]++;
        RFunc(-, pos+1);
        mark[i]--;
    }
}
```

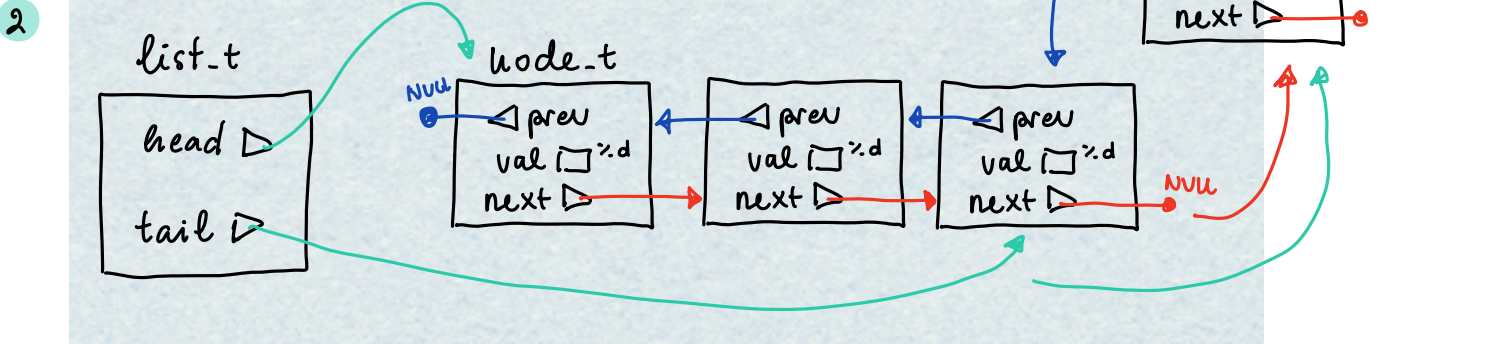
18pt



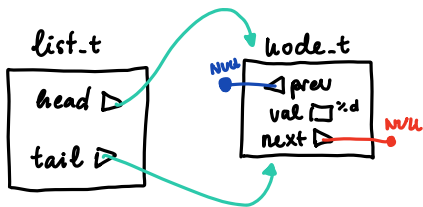


2014-06-27

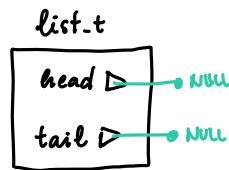
12 pt



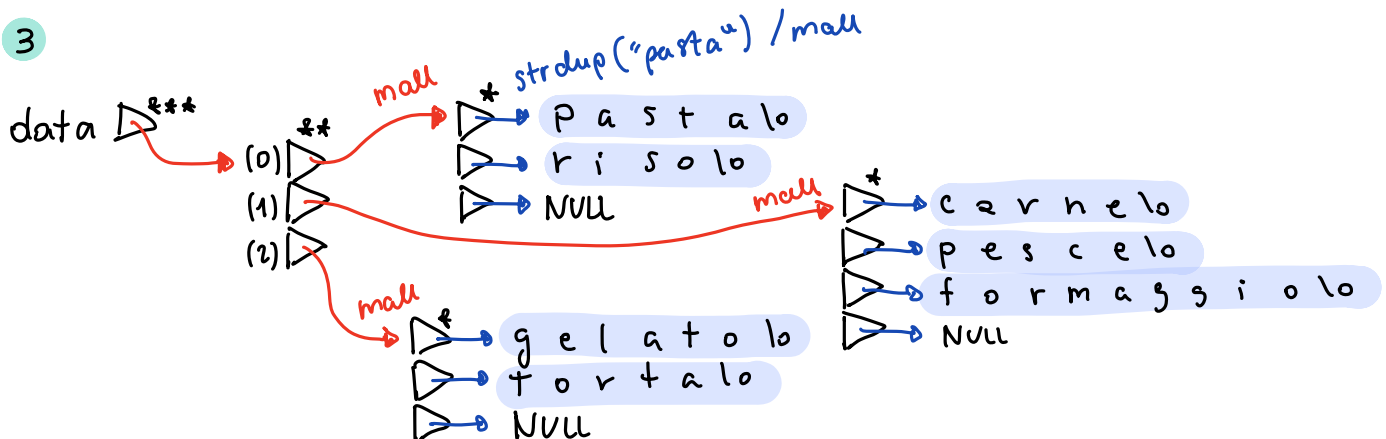
// CASE LIMIT 1

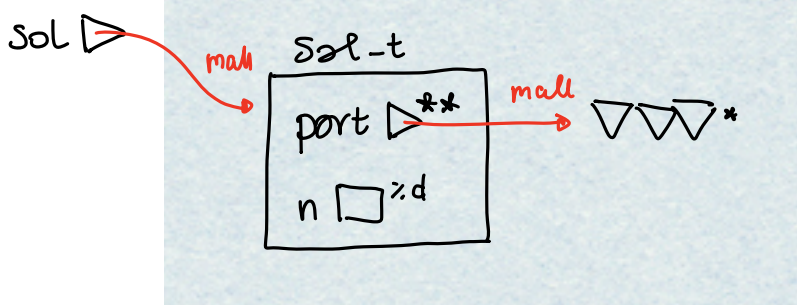


// CASE LIMIT 2

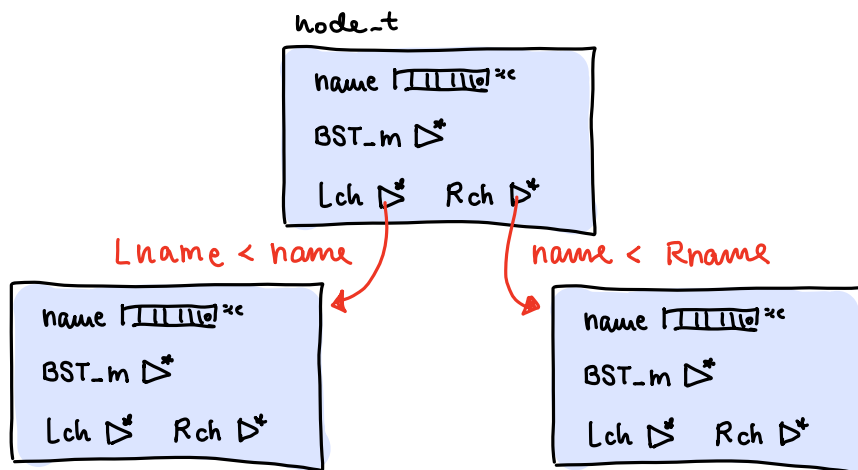
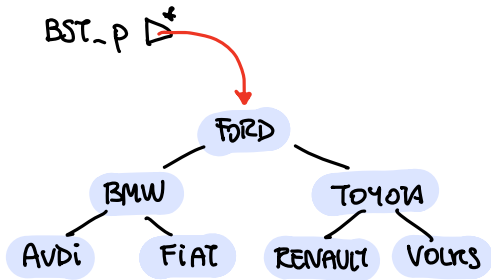


3





18 pt



2014-09-03

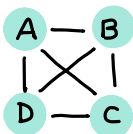
18 pt

sol $\rightarrow$ def

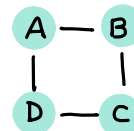
A $\rightarrow$ B	B $\rightarrow$ C	C $\rightarrow$ D	D $\rightarrow$ A
-------------------	-------------------	-------------------	-------------------

 Edge

	A	B	C	D
A		X	X	X
B	X		X	X
C	X	X		X
D	X	X	X	



	A	B	C	D
A		1		1
B	1		1	
C		1		1
D	1		1	



len
K = commi
max

m = deg max

5 | 1 1 1 1 1 1 1

Sol:

1	1	1	1	1	1
---	---	---	---	---	---

	A	B	C	D
A		0	1	2
B			3	4
C				5
D				

Sol:

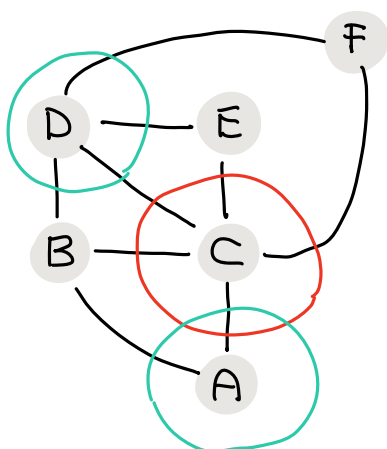
0	1	2	3	4	5
---	---	---	---	---	---

$0 = [0][1]$
 $1 = [0][2]$
 $2 = [0][3]$
 $3 = [1][2]$
 $4 = [1][3]$
 $5 = [2][3]$

	A	B	C	D
A	0	-1	-1	-1
B	-1	0	-1	-1
C	-1	-1	0	-1
D	-1	-1	-1	0

2015-02-02

18 pt



$V = 6$

$E = 9$

$N_{min} = 3$

{Claudia}

{Andrea, Diego}

{Bartolo, Emilia, Fabio}

	A	B	C	D	E	F
A		X	X			
B	X		X	X		
C	X	X		X	X	X
D		X	X		X	X
E			X	X		
F			X	X		

sol_t

$N \square \times d$
 tipo $\triangleright^*$

--	--	--	--	--	--

A B C D E F

ST.e

typedef struct symboltable * ST;

//FUNZIONI di LIBRERIA

ST STinit(int maxN);
 int STinsert(ST st, char *str);
 int STgetindex(ST st, char *str);
 char * STsearchByIndex(ST st, int id);
 void STdisplay(ST st);

ST.c

struct symboltable {

int maxN;
 int n;
 char **v;

}

GRAPH. C

```
typedef struct {int v1, v2;} Edge;
typedef struct graph * Graph;
// FUNZIONI di LIBRERIA
Graph GRAPHinit(int V);
void GRAPHinsertE(Graph G, int id1, int id2, int wt);
static Edge EdgeCreate(int v1, int v2);
static void insertE(Graph G, Edge e);
```

GRAPH.C

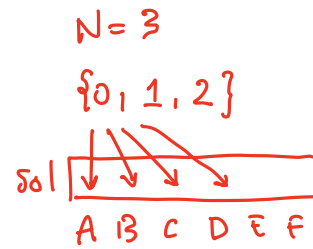
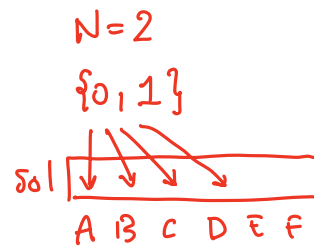
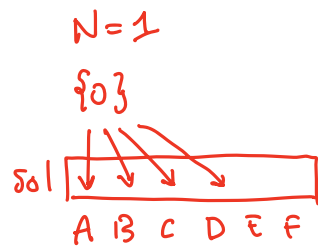
```
struct graph {
    int V, E;
    int ** matAdj;
    ST tableV;
}
```

// n° minimo di partizioni

for(N=1; N<V; N++) {

sol → N = N

↓ RPartizioni(G, sol, pos);

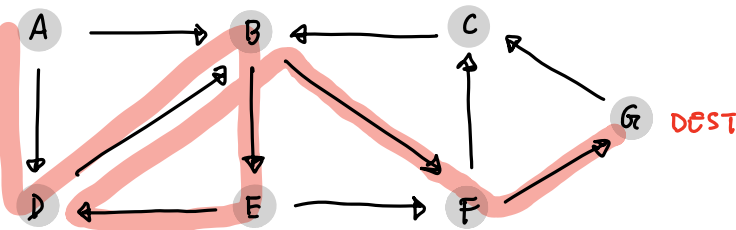


0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1
1 1 2
1 2 0
1 2 1
...

2015-02-23

18 pt

START



sol | 0 3 1 4 3 1 5 6

visited | 1 2 0 2 1 1 1
A B C D E F G

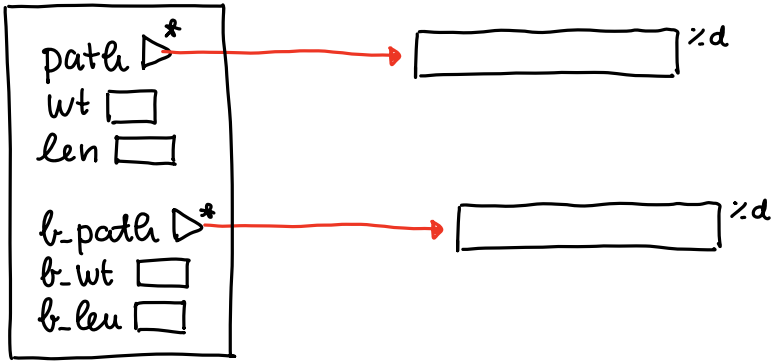
K=2

p=2

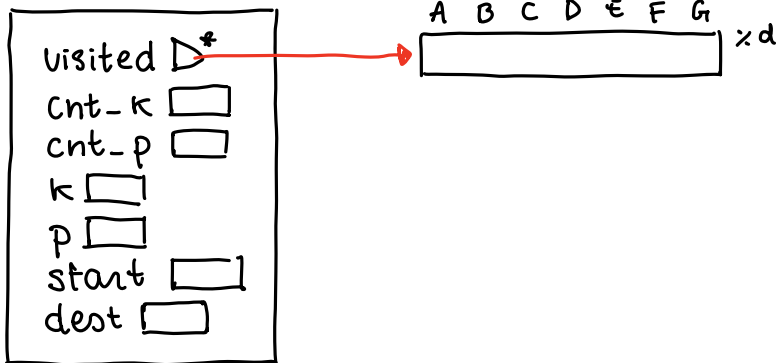
OB = max wt

	A	B	C	D	E	F	G
0	A	X		X			
1	B					X	X
2	C	X					
3	D	X					
4	E			X		X	
5	F		X				X
6	G		X				

sol_t



info_t

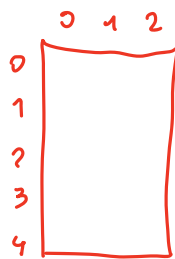
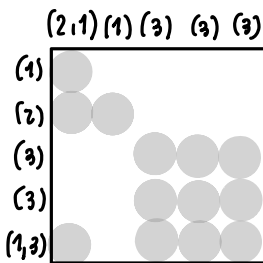


RICORSIONE (node, pos)
if(madj[NODE][j] != 0)
 ↓
RICORSIONE (j, pos+1)

2015-05-02

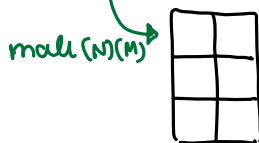
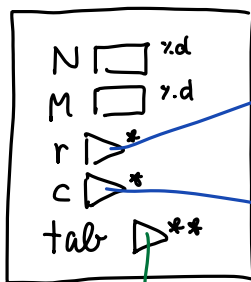
$i \cdot M = 3$ $j \cdot N = 5$

18pt

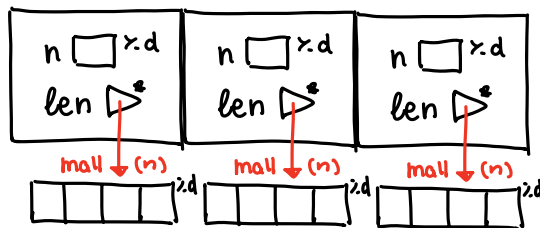


tab[j][i]
0 0
1 0
2 0
3 0
4 0
5 0

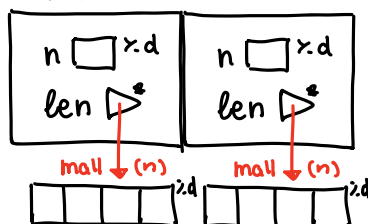
mono_t



info_t



info_t

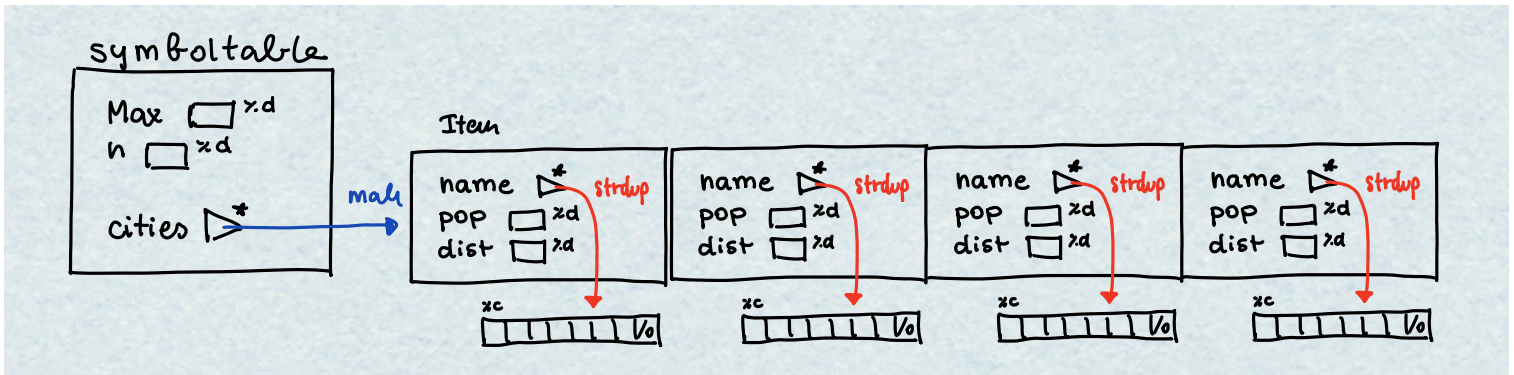


2016-01-26

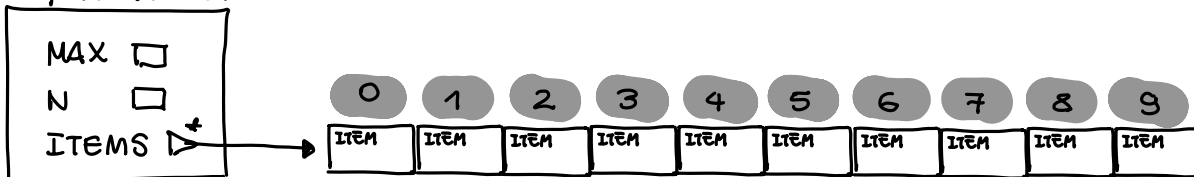
18pt Elenco di città

stringhe $\Rightarrow$ ST !!!

PIACENZA 102 0
FIDENZA 27 35
PARMA 190 57



symboltable



Ogni città è identificata dal suo INDICE in st!

	0	1	2	3	4
0		dist			
1					
2					
3					
4					

dist =

Sol-t

prov sd

best best_sd

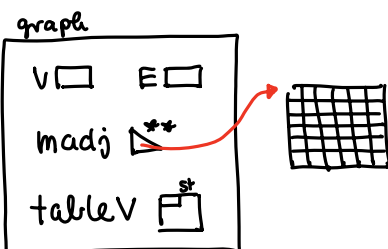
check k

2016-02-22

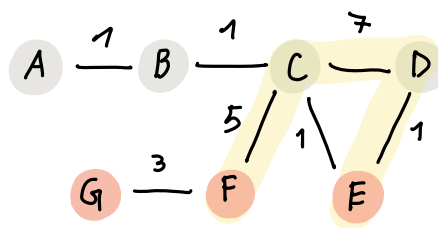
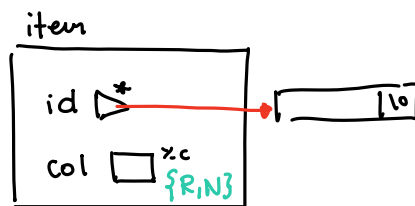
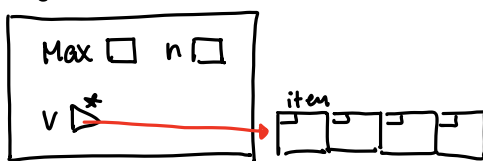


GUARDA
CODICE
RICORSIVO

18pt



st



	A	B	C	D	E	F	G
A		1					
B	1		1				
C		1		7	1	5	
D			7		1		
E			1	1			
F			5				3
G							3

Cammino SEMPLICE $\Rightarrow$ visited A B ... G

CONNESSO $\Rightarrow$ or

PESO MAX $\Rightarrow$ wt ☐ wt_max ☐

$\Rightarrow$ PATH B | A | D ... len ☐

$\Rightarrow$ PATH-MAX [] [] []

$|N^o VR - N^o VN| \leq 2$ len_max ☐

$\hookrightarrow$ R_cnt ☐ N_cnt ☐

2016-09-09

18pt



INSIEME MASSIMA

$\oplus$

COMBINAZIONI SEMPLICI

GR.H

```
typedef struct graph * Graph;
typedef struct { int v1, v2, wt; } Edge;
typedef struct sol {
    int * curr, * best; //elenco di indici di v
    int len_best;      //max - G -> v
    int * done; //flag per non ripet
    // call - G -> v
    //no perché comb semp
    //NoN imp ordine
} sol-t;
```

GR.C

```
struct graph {
    int V, E;
    int ** madj;
    ST tableV;
}
```

ST.H

```
typedef struct symboltable * ST;
```

ST.C

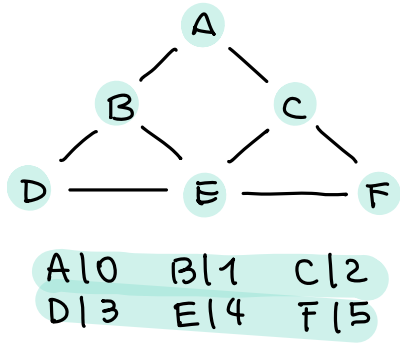
```
struct symboltable {
    /
```

```

int n, Max;
char **V;
3

```

	A	B	C	D	E	F
A		X	X			
B	X			X	X	
C	X				X	X
D		X			X	
E		X	X	X		X
F			X		X	



sol → best 0 3 5 [] [] []

sol → leu_best 3

Sol A D E CHECK Ins Incl

	A	B	C	D	E	F
A		X	X			
B	X			X	X	
C	X				X	X
D		X			X	
E		X	X	X		X
F			X		X	

A: $\text{madj}[A] \begin{cases} \text{madj}[A][D] == 0 \\ \text{madj}[A][E] == 0 \end{cases}$
D: $\text{madj}[D] \rightarrow \text{madj}[D][E] == 0$
E:

INS_MAX &&
COMB-SEMPlici

sol → curr A [] [] [] [] [] [] comb-semplici

sol → curr [] [] [] [] [] [] if (trovato)

sol → curr [] [] [] [] [] [] if (trovato)

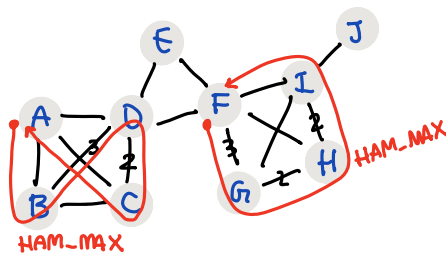
sol → curr [] [] [] [] [] [] if (trovato) → X stop

// for (j = start; j < V; j++)
→ start → j + 1
→ pos → pos + 1

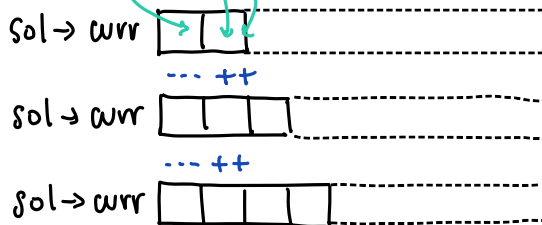
2017_02_04

INSIERE $\Rightarrow$ start → i + 1

SEMPLICI $\rightarrow pos \rightarrow pos+1$



sol $\rightarrow$ len = 2



if(trovato) →
if(trovato) →
if(trovato) →

CRICCHE MASSIMALI

sol $\rightarrow$ CM 



mall. v

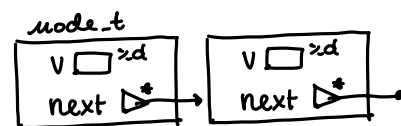

$$\underline{I} \rightarrow J \rightarrow \text{NULL}$$

$D \rightarrow E \rightarrow F \rightarrow \text{NULL}$

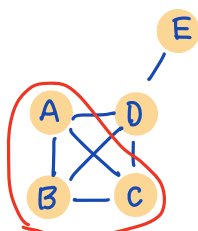
$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow \text{NULL}$$

$F \rightarrow G \rightarrow H \rightarrow I \rightarrow \text{NULL}$

sol $\rightarrow$ n-cm 4



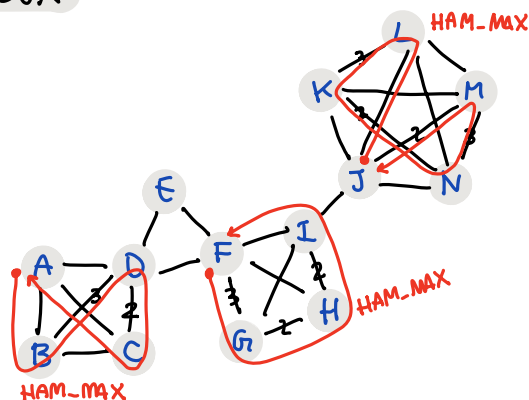
CHECK CRICCA + MASSIMALE



	A	B	C	D	E
A		1	1	1	
B	1		1	1	
C	1	1		1	
D	1	1	1		1
E				1	

Sol $\boxed{A \ B \ C}$
len $\boxed{3}$

PROVA



2017-06-26

18 pt

|| N° PRENOTAZIONI



ST

11. NON NOTO A PRIORI

ST.H

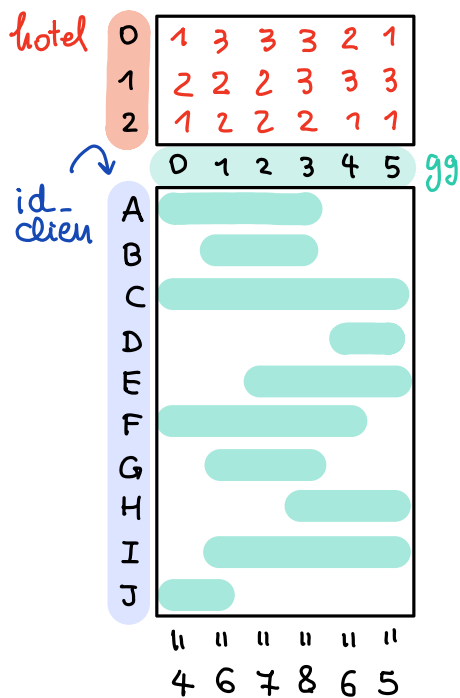
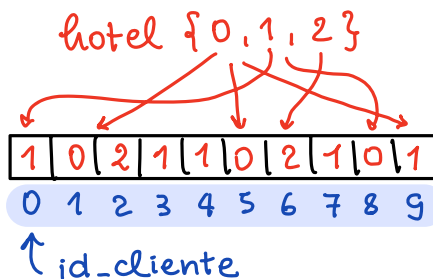
```
typedef struct st * ST;
typedef struct {
    char *id;
    int arrivo;
    int permitt;
    Item;
```

ST.C

```
struct st {
    int n, Max;
    Item *v;
```

FILE2 : sol da verificare

<id-cl hotel> $\Rightarrow$ sol $\rightarrow$ curv



hotel[0] = 7

hotel[1] = 3

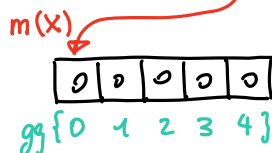
hotel[2] = 5

info.t

```
//n° hotel
N
//vets di hotel
hotel
//n° di gg
X
//coeff
a b
```

hotel.t

```
//n° camere
M
//vets di giorni
//cont il n° di camere occ
h_camocc-gg
//n° camere
M
//vets di giorni
//cont il n° di camere occ
h_camocc-gg
//n° camere
M
//vets di giorni
//cont il n° di camere occ
h_camocc-gg
```



2017-09-18

18 pt

koala : $K_0 \dots K_{(N-1)}$
famiglie : $F_0 \dots F_{(S-1)}$
eucalipti : $0 \dots (T-1)$

$$m = n^0 \max R \text{ per } E$$

koala-t

id $\square \geq d$
family $\square \geq d$
 $n \square \rightarrow$ $m(fsc) \square \geq d$
euc-or $\rightarrow$ $\square \geq d$

sol-t

The diagram illustrates the recursive step of the Longest Common Subsequence (LCS) algorithm. It shows two arrays, `curr` and `best`, both of size $m(n)$. The `curr` array is updated based on the `best` array. Blue arrows indicate the recursive calls for the subproblems $k_0, k_1, k_2, k_3, k_4, k_5$.

min_occ $\boxed{}$ $\approx d$
init $\gg T$

len $\geq d$
init $\gg N$

F0: 0 1 2

F1: 3 4

f2: 5 6

f3: 7 8 9

F4: 10 11

	0	1	2	3	4
0			X		X
1				X	X
2	X				X
3		X			X
4	X	X	X	X	

EO 034

E1 1 7 8

$$E_2 \quad 29$$

10 11

E4 5 6

E5

E6

sol $\rightarrow$ wrr

0	1	2	0	0	4	4	1	1	2	3	3
---	---	---	---	---	---	---	---	---	---	---	---

k: 0 1 2 3 4 5 6 7 8 9 10 11

info-t

N	☐	%d
S	☐	%d
T	☐	%d
m	☐	%d

koalas $\xrightarrow{m(n)}$ koala.t

Diagram illustrating the state space for a 5x5 game. The grid shows states (s, s) marked with red 'X's. A red arrow labeled $m(s, s)$ points from the top-left state (0,0) to the top-right state (0,4). Another red arrow labeled $m(T)$ points from the top-right state (0,4) to the bottom-left state (4,0). The grid is labeled with indices 0 to 4 on both axes.

init $\gg 0, 0, 0$

2018-01-29

18pt

0 | "AnnaAstri"
1 | "BeaBianchi"
2 | "ClarkCotti"
3 | "DanDitto"

1 ↓ 2 ↓ for X

1	3	0	2
2	0	3	1
1	2	0	3
3	0	2	1

0 | "Altar"
1 | "Biotech"
2 | "Crios"
3 | "Dyson"

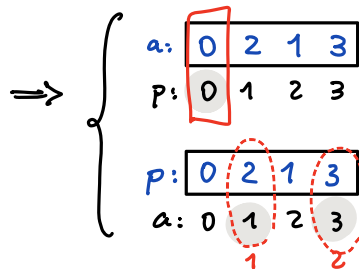
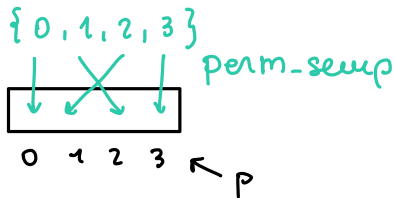
1	0	3	2
3	2 ^x	0	1
0	3	2	1
1	0	3 ^x	2

inst(0,1)? X
inst(0,3)? ✓

⇒ ST

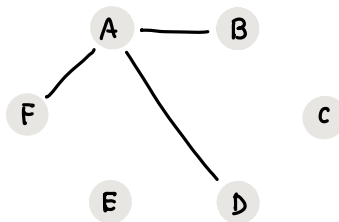
INSTABILITA': $p_i \text{ pref } a_i \text{ \&\& } a_i \text{ pref } p_i$

SOL:



2018-02-13

18pt



diametro
connessione

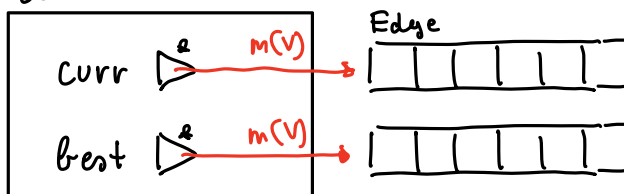
	A	B	C	D	E	F
A	X			X		X
B	X	X				
C			X			
D	X			X		
E					X	
F	X					X

$$n^{\circ}E = \frac{V \cdot V - V}{2} - E$$

OB: aggiungere il n° minimo (INS. A CARDINALITA' MIN) di archi
⇒ grafo connesso

SOL OTTIMA: DIAMETRO MINIMO

sol-t

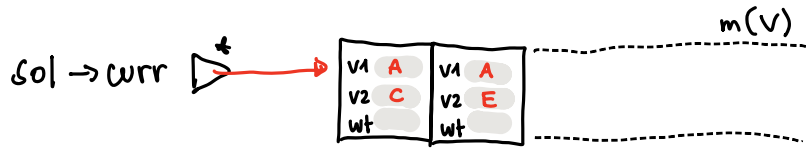
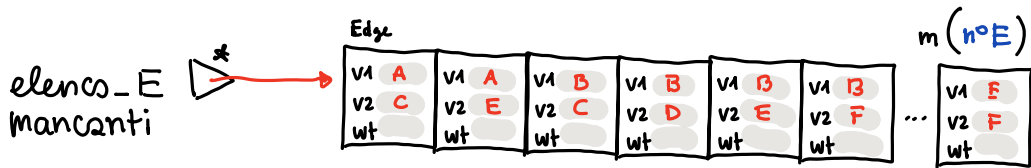


// con V archi sicuramente
grafo connesso!!!

d_min %d
len %d

// >> INT_MAX

// >> 1 → 2 → 3 finché non connemo

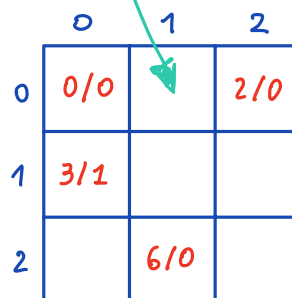
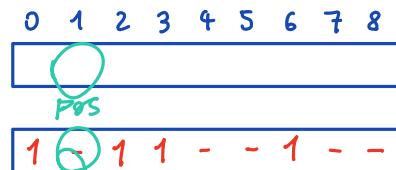
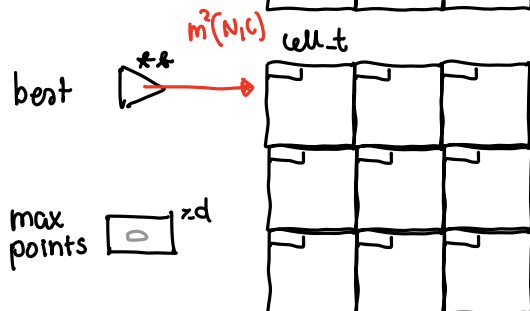
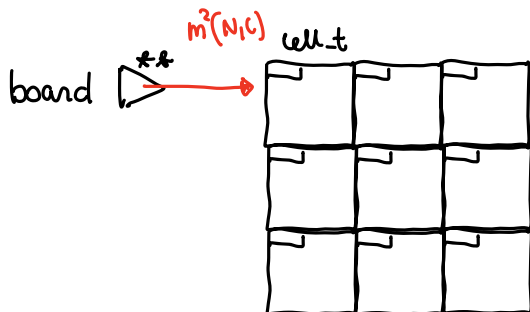
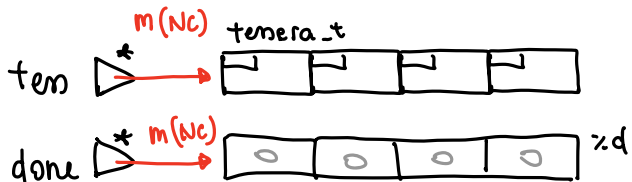
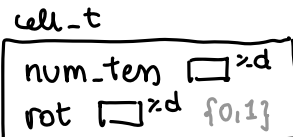
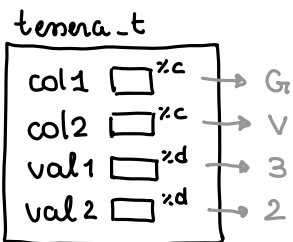
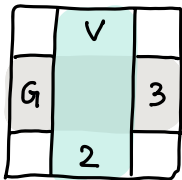


ORDINE NON IMPORTANTE
NO RIPETIZIONI

COMB. SEMPLICI

2018-06-21

18pt



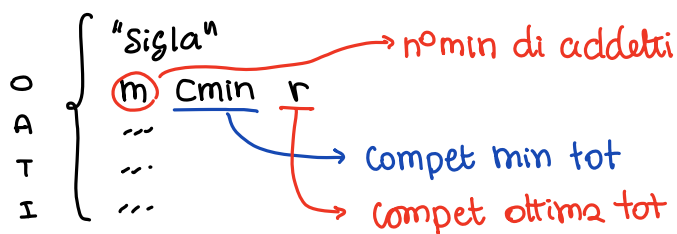
$i = pos / N;$
 $j = pos \% C;$

2019_02 - 22

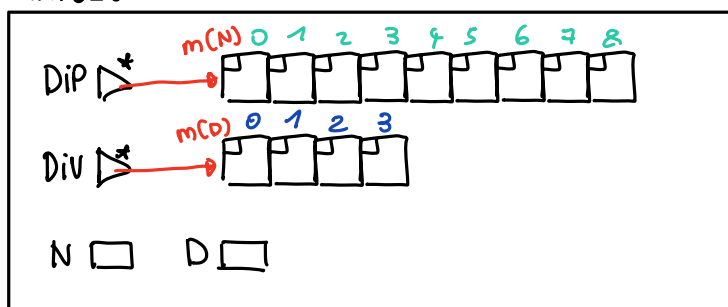
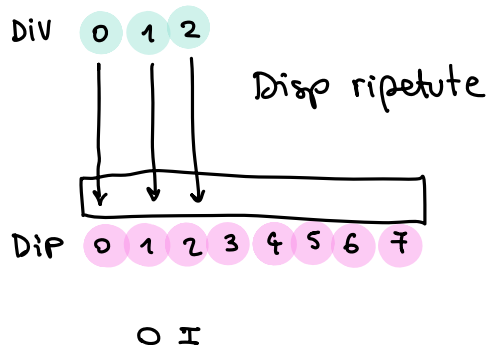
18pt 111 Manio Rossi 10 60 30 80

$N = n^{\circ}$ dipendenti

$D = n^{\circ}$ divisioni

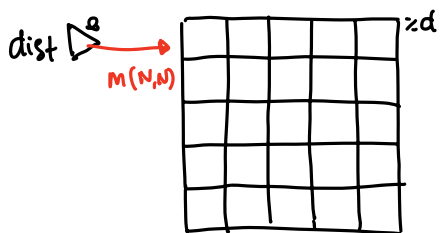
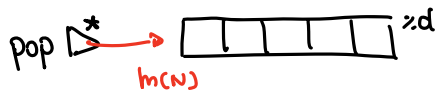


Sæl <mat> <ruolo> <sigla>



2019_07_02

18pt $N = \text{n}^\circ \text{ comuni}$

$$N = \text{n}^\circ \text{ comuni}$$


$$f_1: \begin{cases} \text{distMax } \boxed{}^{zd} \\ \downarrow \\ \text{statMax } \boxed{}^{zd} + \text{localizer} \end{cases}$$

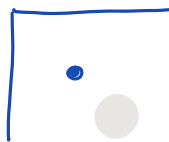
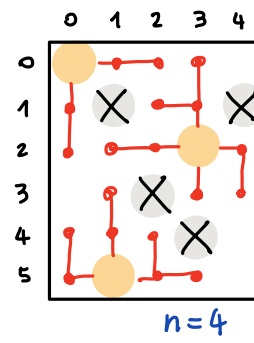
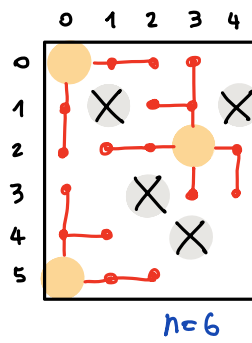
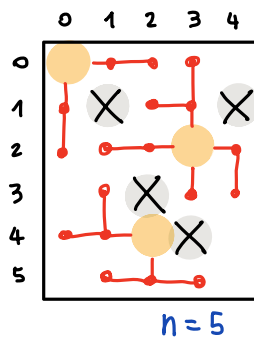
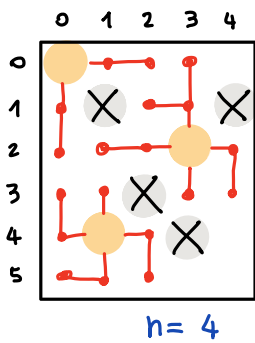
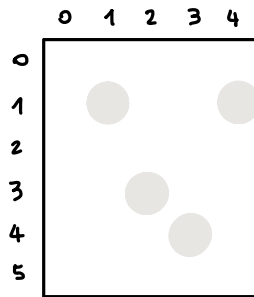
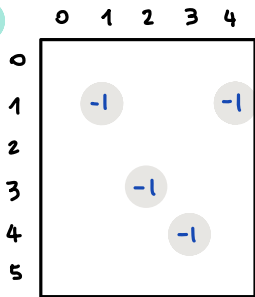
$$fz: \begin{cases} \text{numStat } \boxed{}^{x.d} + \text{statConn } \boxed{} \boxed{} \boxed{} \boxed{} \boxed{}^{x.d} \\ \Downarrow \\ \text{localizer} \quad \text{t.c. MIN } \{ \text{Funs} \} \end{cases}$$

Ex: Ins card min $\square \rightarrow \boxed{\square} \rightarrow \boxed{\square \square} \dots$ + VERIFICATION FINALE

com { 0, 1, 2, 3, 4, 5 }
 ↓
 sol []
 cnt [1 1]

2019-09-18

18pt



start i
~~for~~ (int $i=0$; $i < N$; $i++$) {
 for (int $j=0$; $j < N$; $j++$) {

if (info.map[i][j] != -1 && sel → currMap[i][j] == 0) {

sel → curr[pos][0] = i;
 sel → curr[pos][1] = j;

sel → currMap[i][j] = pos + 1;
 RecolorMap (info, sel, i, j, pos + 1);

RecurSel (info, sel, i, j, pos + 1, i, j);

DecolorMap (info, sel, i, j);